



컴퓨터 그래픽스

10. 2D 게임과 애니메이션

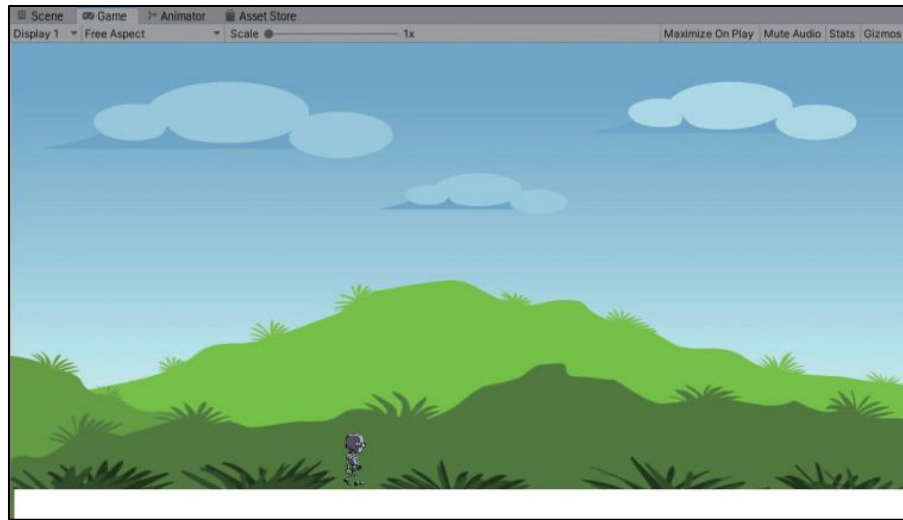
1. 프로젝트 생성과 배경 오브젝트 배치
2. 에셋 가져오기
3. 2D 애니메이션 설정
4. 폭탄 피하기
5. UI 설정
6. 애니메이터 컨트롤러와 상태 머신
7. 씬 전환
8. 빌드 생성

상명대
소프트웨어학과
배동성

학습 목표

- 유니티에서 2D 게임을 만드는 방법을 익힌다.
- 2D 애니메이션 클립을 만드는 방법을 익힌다.
- 애니메이터 컨트롤러를 설정하는 방법을 익힌다.
- 씬을 전환하는 방법을 익힌다.
- 빌드를 만드는 방법을 익힌다.

1. 개요



- 개발 대상이해: 2D 횡스크롤 게임의 간략한 버전
- 리소스 준비: 배경이미지, 캐릭터 등 필요



배경



로봇



그라운드



폭탄



금화



바위

- 프로그래밍 요소 파악:
 - 캐릭터 이동, 충돌 처리, 점수 구현
- 유니티 설정 요소
 - 애니메이션 설정

2. 프로젝트 생성과 배경 오브젝트 배치

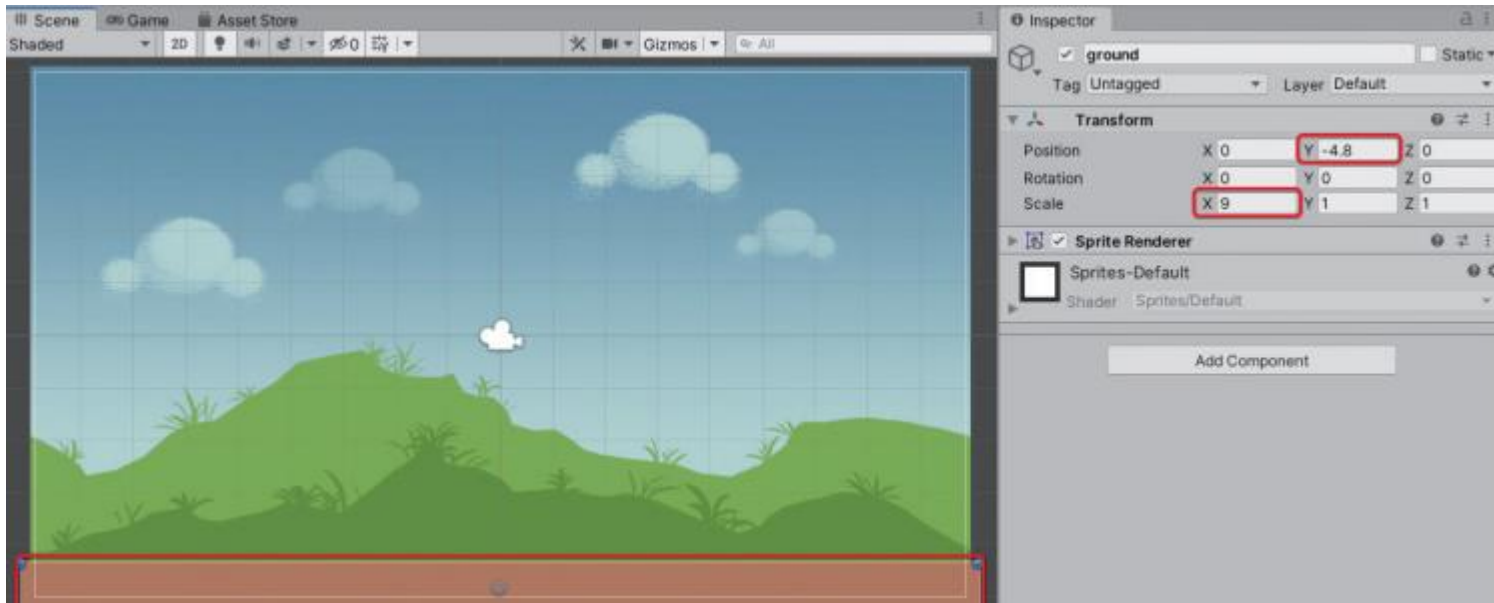
- 프로젝트 생성 및 배경 오브젝트 배치: **2D**, RobotRun01



배경이미지 스케일 조정

2. 프로젝트 생성과 배경 오브젝트 배치

- 그라운드 이미지 배치

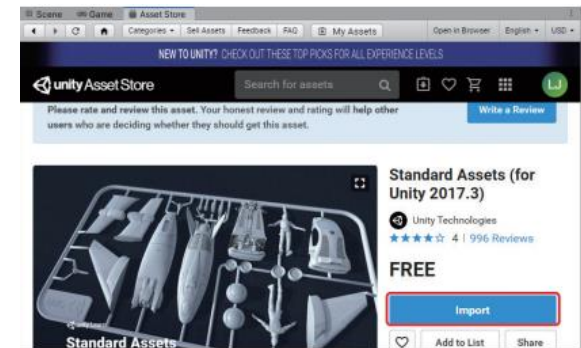


그라운드 이미지
위치와 스케일 조정

3. 에셋 가져오기

- Standard Assets 가져오기

- [Window]-[Asset Store]를 선택하여 에셋 스토어에 접속
- 'Standard Assets'를 검색하여 다운로드한 후 импорт
- [Project] 창에서 Standard Assets 폴더 확인



- Standard Assets 오류 수정



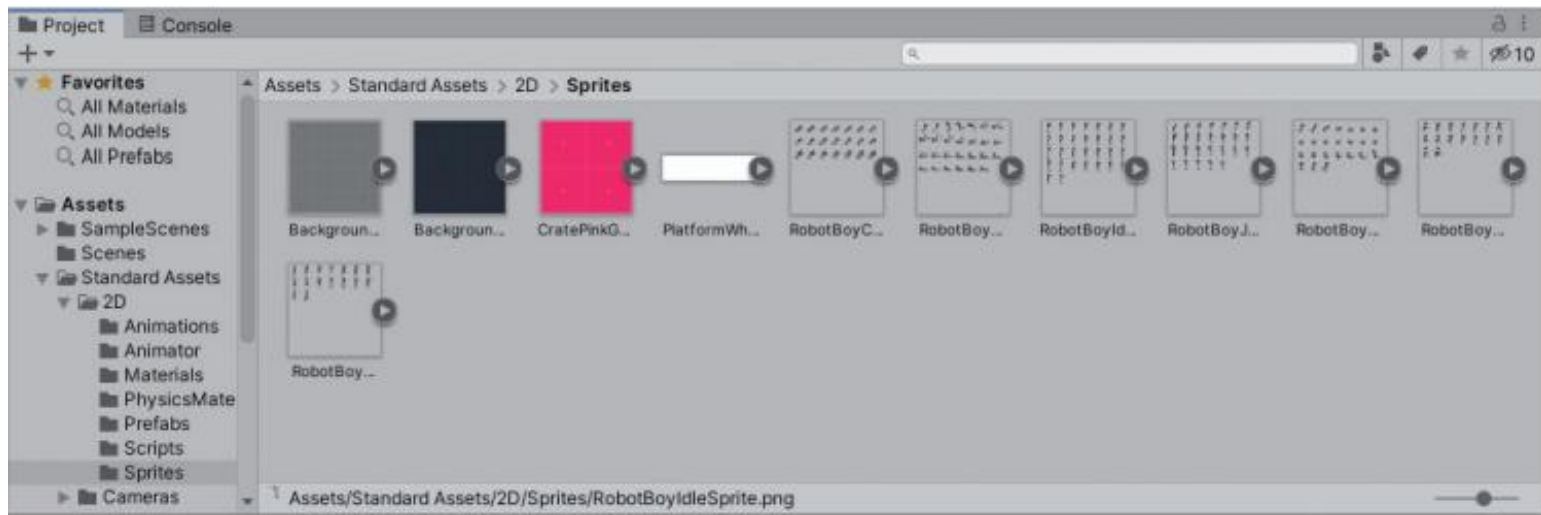
```
1 using System;
2 using UnityEngine;
3 using UnityEngine.UI;
4
5 #pragma warning disable 618
6 namespace UnityStandardAssets.Utility
7 {
8     public class SimpleActivatorMenu : MonoBehaviour
9     {
10         // An incredibly simple menu which, when given references
11         // to gameobjects in the scene
12         public Text camSwitchButton;
```

UnityEngine.UI 추가

GUIText -> Text

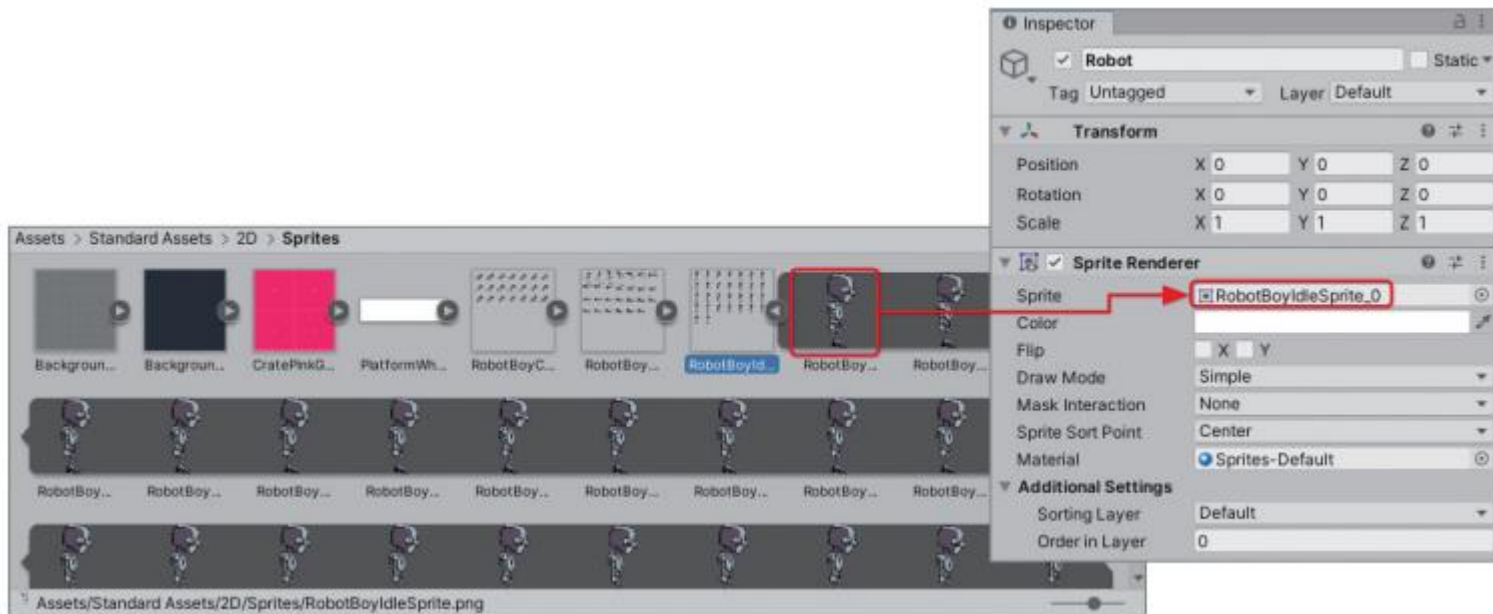
3. 에셋 가져오기

- 캐릭터로 사용할 로봇 이미지 가져오기
 - 2D 이미지를 스프라이트라고 함
 - Standard Assets->2D->Sprites 폴더



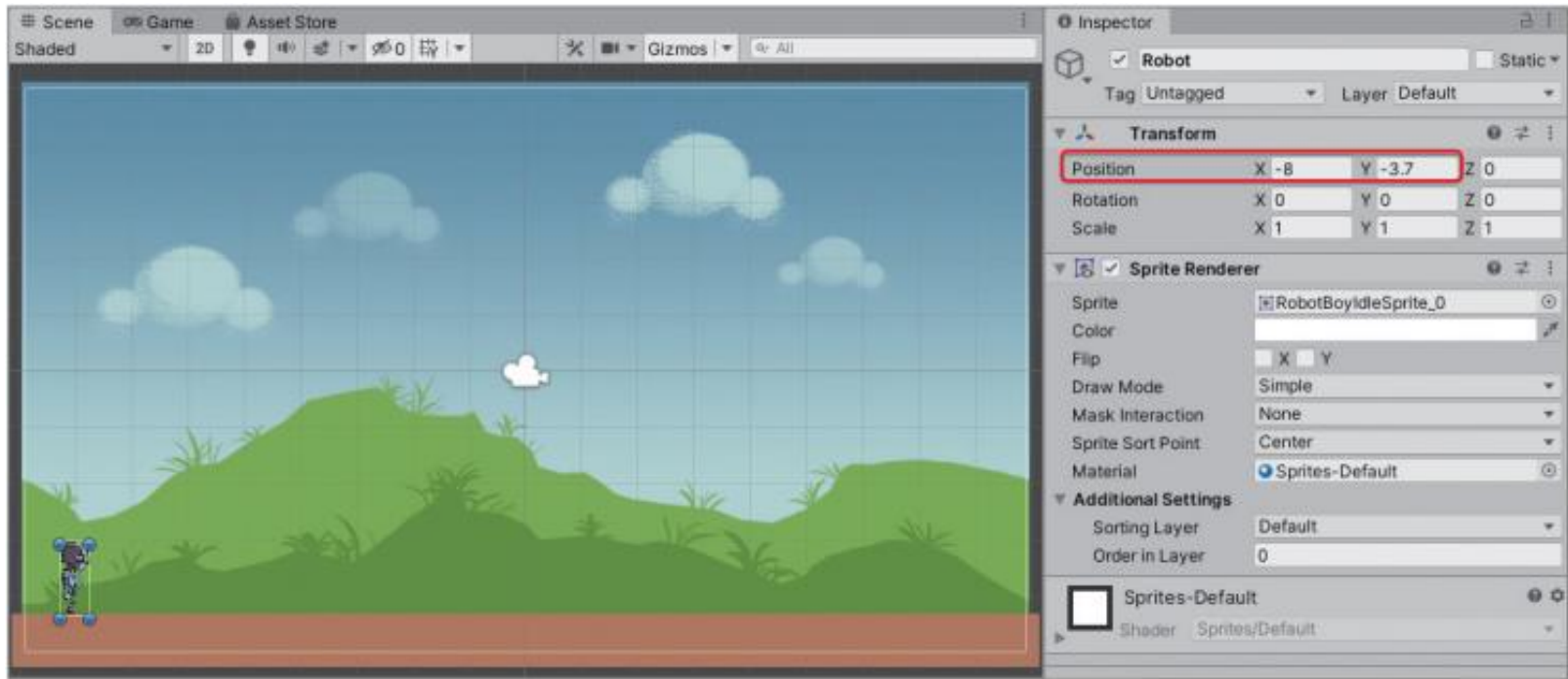
3. 에셋 가져오기

- RobotBoyleSprite.png 파일을 선택하여 오른쪽의 삼각형을 클릭
- 이중 RobotBoyleSprite_0 스프라이트를 선택



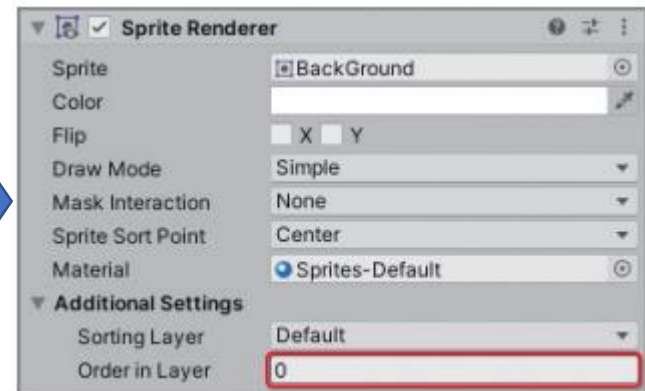
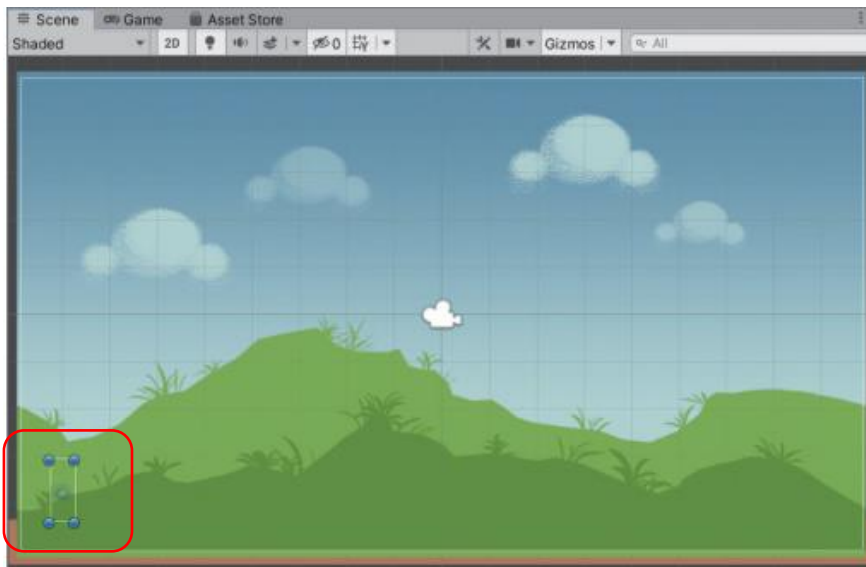
3. 에셋 가져오기

- RobotBoyIdleSprite_0 스프라이트를 끌어다 놓으면 씬에 로봇이 등장
 - 로봇의 Position을 X는 -8, Y는 -3.7로 조정



3. 에셋 가져오기

- 레이어 조정(로봇이 안보일 경우)
 - [Inspector] 창에서 Sprite Renderer 컴포넌트의 Order in Layer 속성 값으로 레이어를 조정
 - 숫자가 클수록 전면에 배치되므로 배경 이미지를 0, 그라운드와 로봇 이미지를 1로 설정



로봇이 안보임
그라운드도 안보임

3. 에셋 가져오기

- 그라운드와 로봇에 물리와 콜라이더 설정
 - 그라운드: Box Collider 추가
 - 로봇: Rigidbody2D, Box Collider 추가

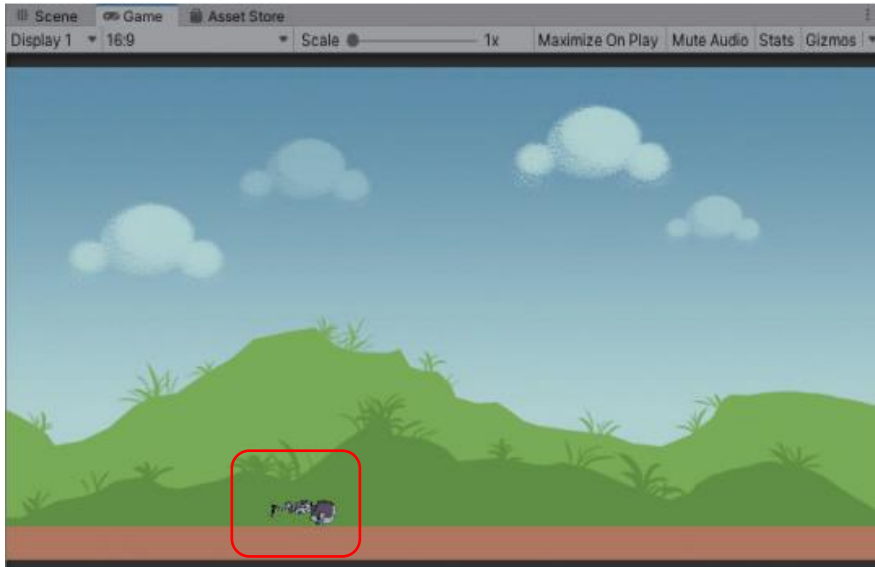
3. 예셋 가져오기

- 로봇 이동 스크립트 작성: RobotController

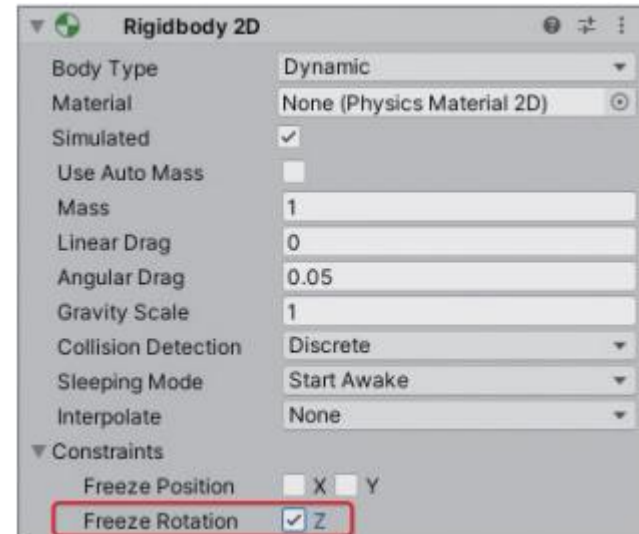
```
01 public class RobotController : MonoBehaviour
02 {
03     Rigidbody2D robotRD;
04     float walkSpeed = 10.0f;
05
06     // Start is called before the first frame update
07     void Start()
08     {
09         robotRD = GetComponent<Rigidbody2D> ();
10     }
11
12     // Update is called once per frame
13     void Update()
14     {
15         if (Input.GetKey (KeyCode.LeftArrow) == true)
16         {
17             robotRD.AddForce (transform.right*-1*walkSpeed);
18         }
19         if (Input.GetKey (KeyCode.RightArrow) == true)
20         {
21             robotRD.AddForce (transform.right*walkSpeed);
22         }
23     }
24 }
```

3. 예셋 가져오기

- 로봇이 넘어진다면



이동중에 넘어짐
z축을 기준으로 회전하는 현상



z축 회전 Freeze

3. 예셋 가져오기

- 로봇의 방향 바꾸기
 - 로봇의 방향 회전: transform.localScale 값을 조정

```
12    // Update is called once per frame
13    void Update()
14    {
15        if (Input.GetKey (KeyCode.LeftArrow) == true)
16        {
17            robotRD.AddForce (transform.right*-1*walkSpeed);
18            transform.localScale = new Vector3 (-1.0f , 1.0f , 1);
19        }
20        if (Input.GetKey (KeyCode.RightArrow) == true)
21        {
22            robotRD.AddForce (transform.right*walkSpeed);
23            transform.localScale = new Vector3 (1.0f , 1.0f , 1);
24        }
25    }
```

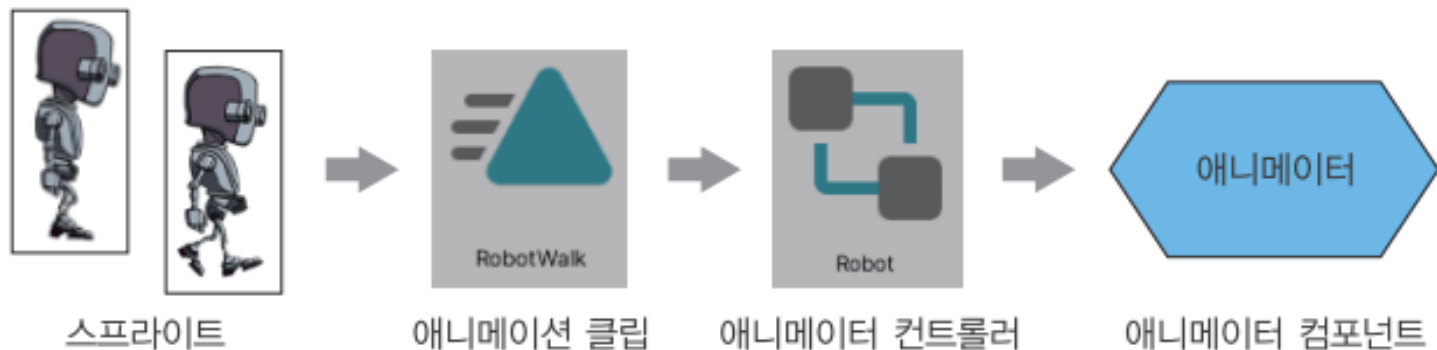
4. 2D 애니메이션 설정

- 유니티 애니메이션
 - 로봇이 자연스럽게 이동하는 것처럼 보이게 하려면 애니메이션 필요
 - 유니티의 2D 애니메이션은 이미지 스프라이트를 일정 간격으로 바꾸는 방식
 - 유니티 애니메이션 구현 방식 2가지
 - 메카닉 사용
 - 스크립트 작성

4. 2D 애니메이션 설정

- 메카닉

- 일종의 상태머신
- 메카닉의 구성요소

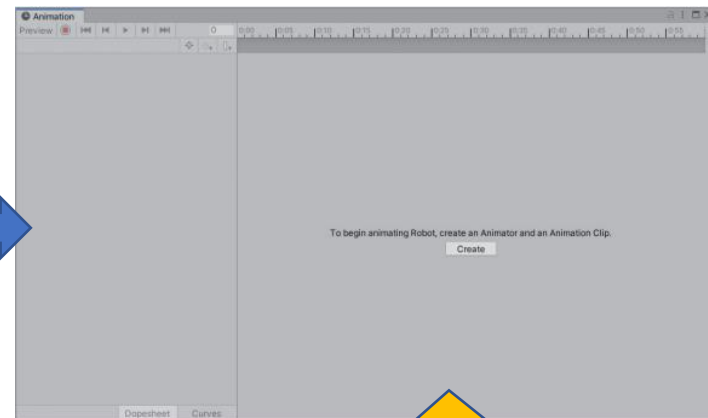
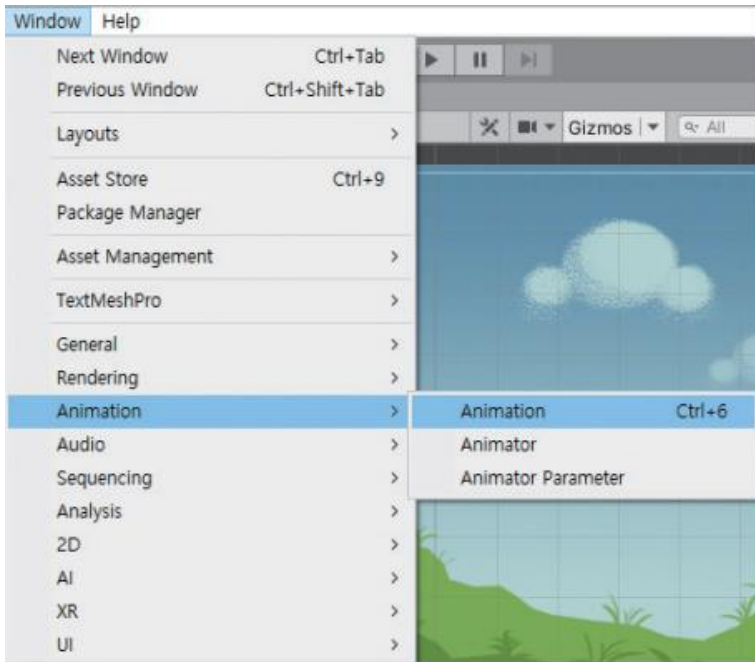


- 애니메이션 클립: 스프라이트 연결 순서, 재생속도, 시간정보 등 동작 정보
- 애니메이터 컨트롤러: 각각의 애니메이션 클립을 모아서 오브젝트가 언제 어떤 애니메이션을 재생할 것인지를 설정
- 애니메이터 컴포넌트: 애니메이터 컴포넌트에 애니메이터 컨트롤러를 설정해야 애니메이션을 재생

4. 2D 애니메이션 설정

• 걷기 애니메이션 설정

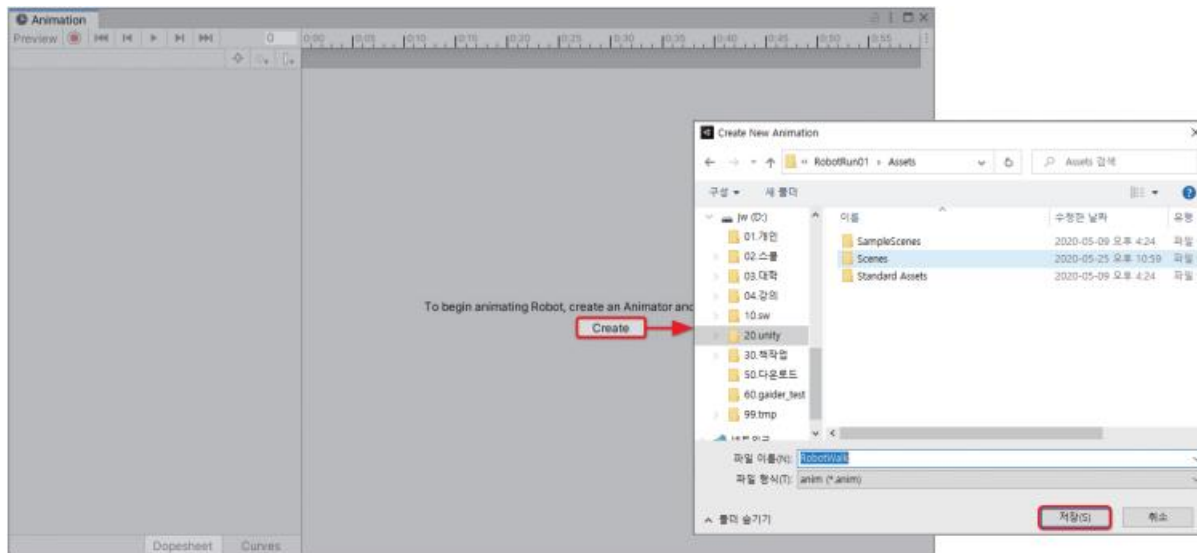
- [Project] 창에서 [Assets]-[Standard Assets]-[2D]-[Sprites]를 선택
- RobotBoyWalkSprite를 클릭하면 16개의 스프라이트가 있는데 여기서 4개 정도를 골라서 애니메이션 생성
- 애니메이션 클립 생성: [Window]-[Animation]-[Animation] 선택



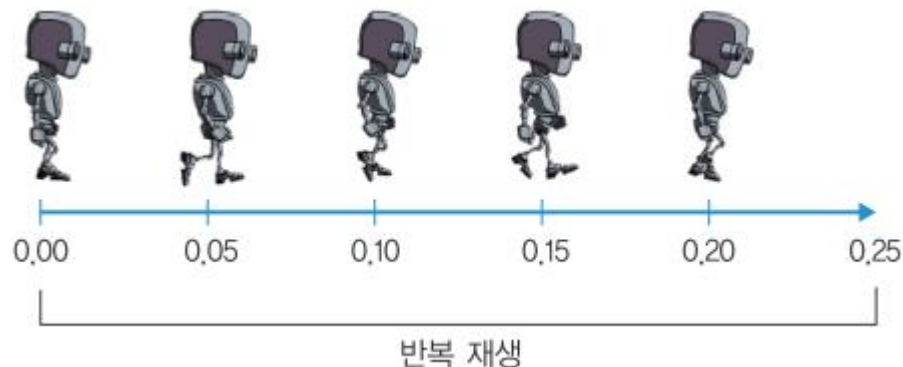
애니메이션창

4. 2D 애니메이션 설정

- [Animation] 창에서 <Create>를 클릭하고 파일명 지정

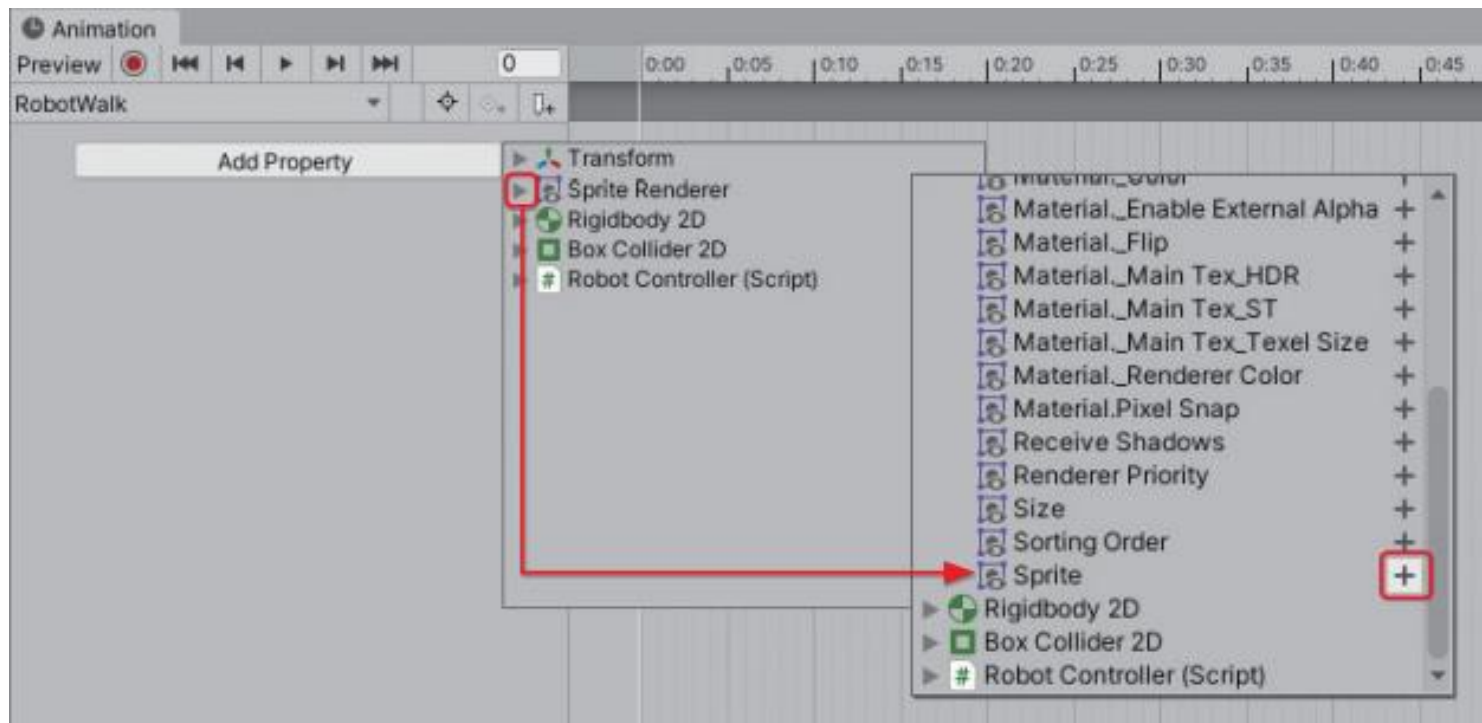


- 로봇이 서 있는 상태의 스프라이트를 포함하여 총 5개의 스프라이트를 0.05초 간격으로 배치



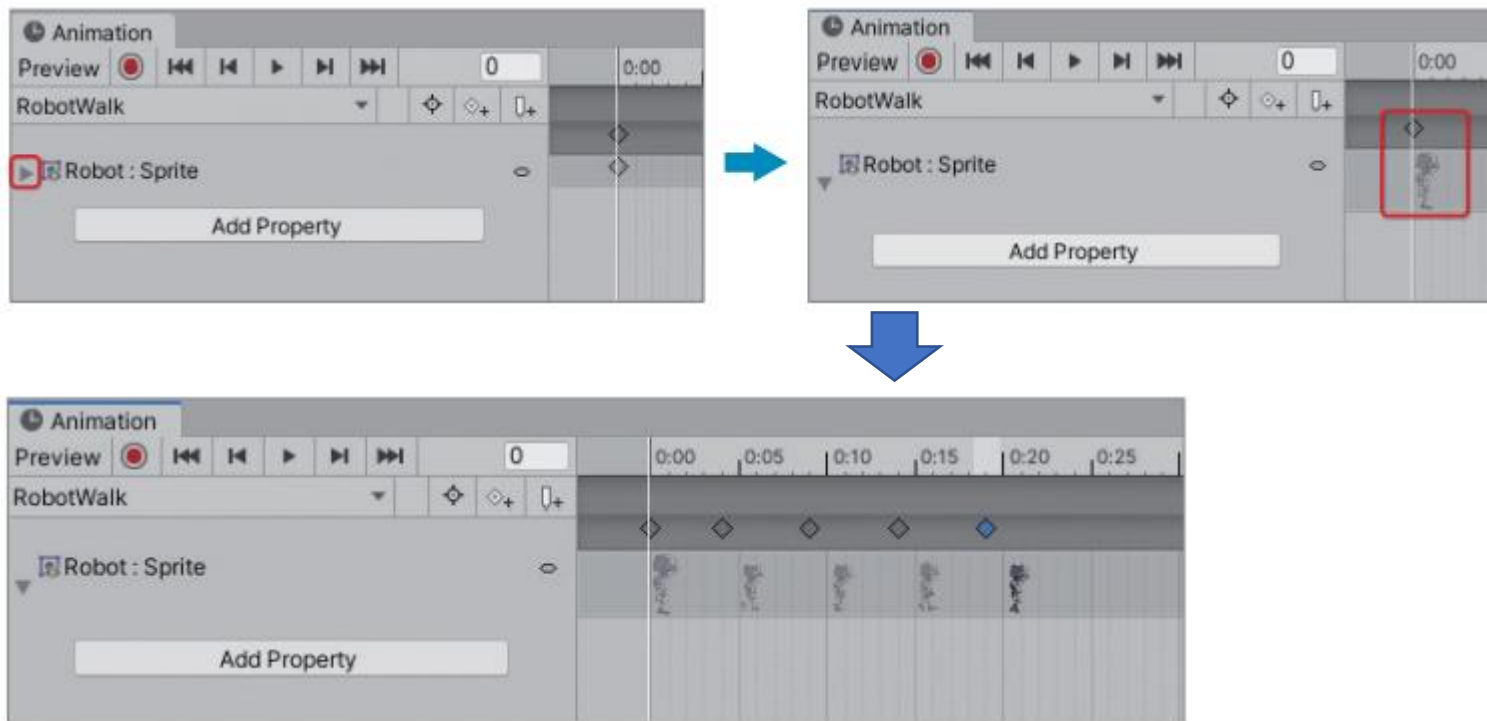
4. 2D 애니메이션 설정

- [Animation] 창의 <Add Property>를 클릭
- Sprite Renderer 옆의 삼각형을 클릭->Sprite의 + 클릭



4. 2D 애니메이션 설정

- Sprite의 +를 클릭하면 기본 스프라이트가 추가
- Robot : Sprite 옆의 삼각형을 클릭하면 타임라인에 스프라이트가 표시
- 이후 다른 스프라이트를 타임라인에 배치

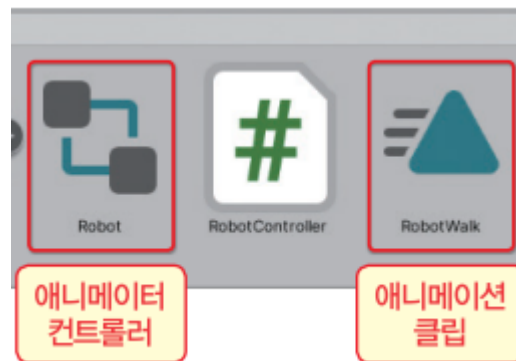
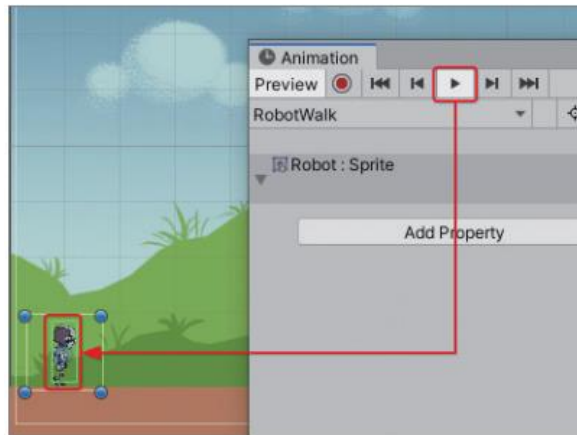


4. 2D 애니메이션 설정

- 스프라이트 배치 완료 후 총 재생 시간인 0.25초 위치를 클릭한 다음 Add Keyframe 아이콘을 클릭
 - 1초 위치에 있는 스프라이트는 삭제



- 재생 버튼을 클릭하여 동작 확인



[Project] 창에서
애니메이션
클립애니메이터
컨트롤러파일
확인

4. 2D 애니메이션 설정

- 로봇 이동속도 조정하기

- AddForce() 함수를 사용했기 때문에 방향키를 계속 누르고 있으면 힘이 계속 가해져서 이동 속도가 점점 빨라짐
- 최대 이동 속도를 지정하여 이를 넘지 않을 때만 힘을 가하도록 수정

```
04     Animator animator;  
05     float walkSpeed = 10.0f;  
06     float maxSpeed = 3.0f;
```

```
19         float speedx = Mathf.Abs (robotRD.velocity.x);  
20  
21         if (speedx < maxSpeed)  
22         {  
23             if (Input.GetKey (KeyCode.LeftArrow) == true)  
24             {  
25                 robotRD.AddForce (transform.right * -1 * walkSpeed);  
26                 transform.localScale = new Vector3 (-1.0f , 1.0f , 1);  
  
34                 animator.speed = speedx;  
35             }  
}
```

5. 폭탄 피하기

- 폭탄 배치 및 충돌처리
 - 폭탄 이미지 배치, Rigidbody2D, Box Collider 추가



```
01 public class bombController : MonoBehaviour
02 {
03     void OnCollisionEnter2D(Collision2D Col)
04     {
05         Destroy (gameObject);
06     }
07 }
```

충돌하면 폭탄
삭제하도록 코딩

5. 폭탄 피하기

- 폭탄 프리팹 만들기

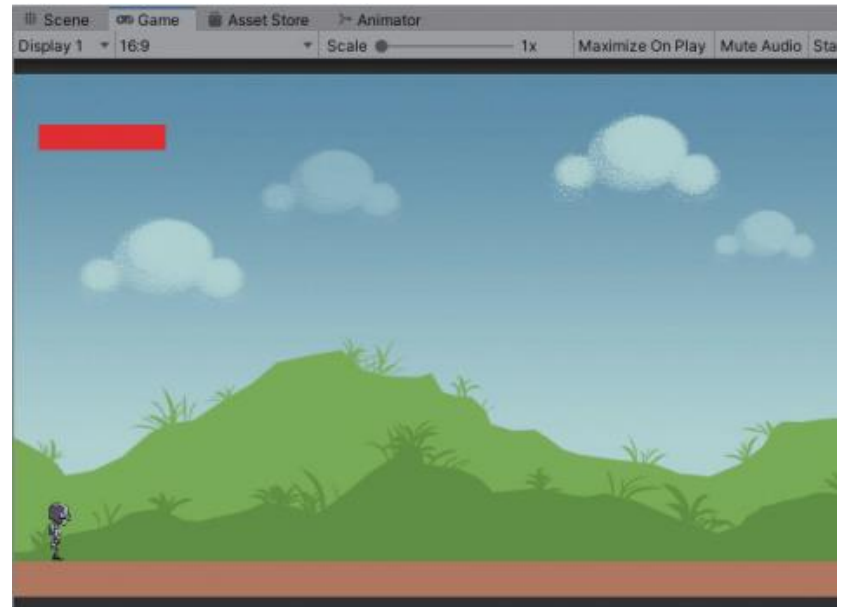
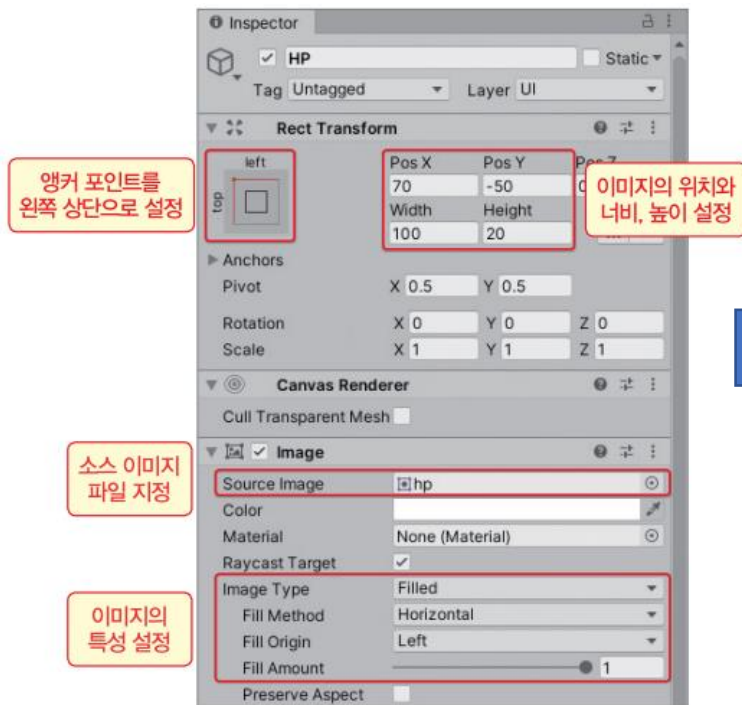


폭탄이 떨어지는 위치를
랜덤으로
조정

```
01 public class bombMaker : MonoBehaviour
02 {
03     public GameObject bmPrefab;
04     float interval = 1.0f;
05     float delta = 0;
06
07     // Update is called once per frame
08     void Update()
09     {
10         delta += Time.deltaTime;
11         if(delta > interval)
12         {
13             delta = 0;
14             GameObject bomb = Instantiate (bmPrefab);
15             int x = Random.Range (-8 , 9);
16             bomb.transform.position = new Vector3 (x , 6 , 0);
17         }
18     }
19 }
```


6. UI 설정

- HP UI 설치
 - HP용 이미지 준비
 - [Hierarchy] 창에서 +를 클릭하고 [UI]-[Image]를 선택
 - 앵커포인트, 크기 지정, 이미지 파일 지정



6. UI 설정

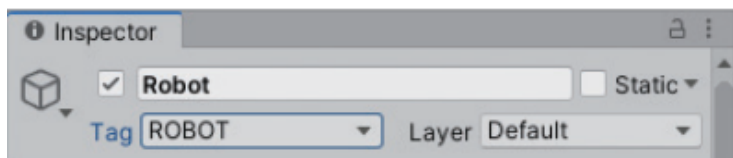
- UI 스크립트 작성: hpController

```
04 using UnityEngine.UI;
05
06 public class hpController : MonoBehaviour
07 {
08     GameObject hp;
09
10     // Start is called before the first frame update
11     void Start()
12     {
13         hp = GameObject.Find ("HP");
14     }
15
16     public void HpControl()
17     {
18         hp.GetComponent<Image> ().fillAmount -= 0.05f;
19     }
20 }
```

HP이미지 크기를
0.05씩 줄이기

6. UI 설정

- 폭탄이 로봇과 충돌할 경우 hpController의 HpControl() 함수를 호출하여 HP 이미지를 조정하도록 bombController 스크립트를 수정
 - 폭탄이 충돌한 대상이 로봇인지 구분할 수 있게 태그 처리



충돌한 대상이 ROBOT이면
hpController의 HpControl()
함수를 호출

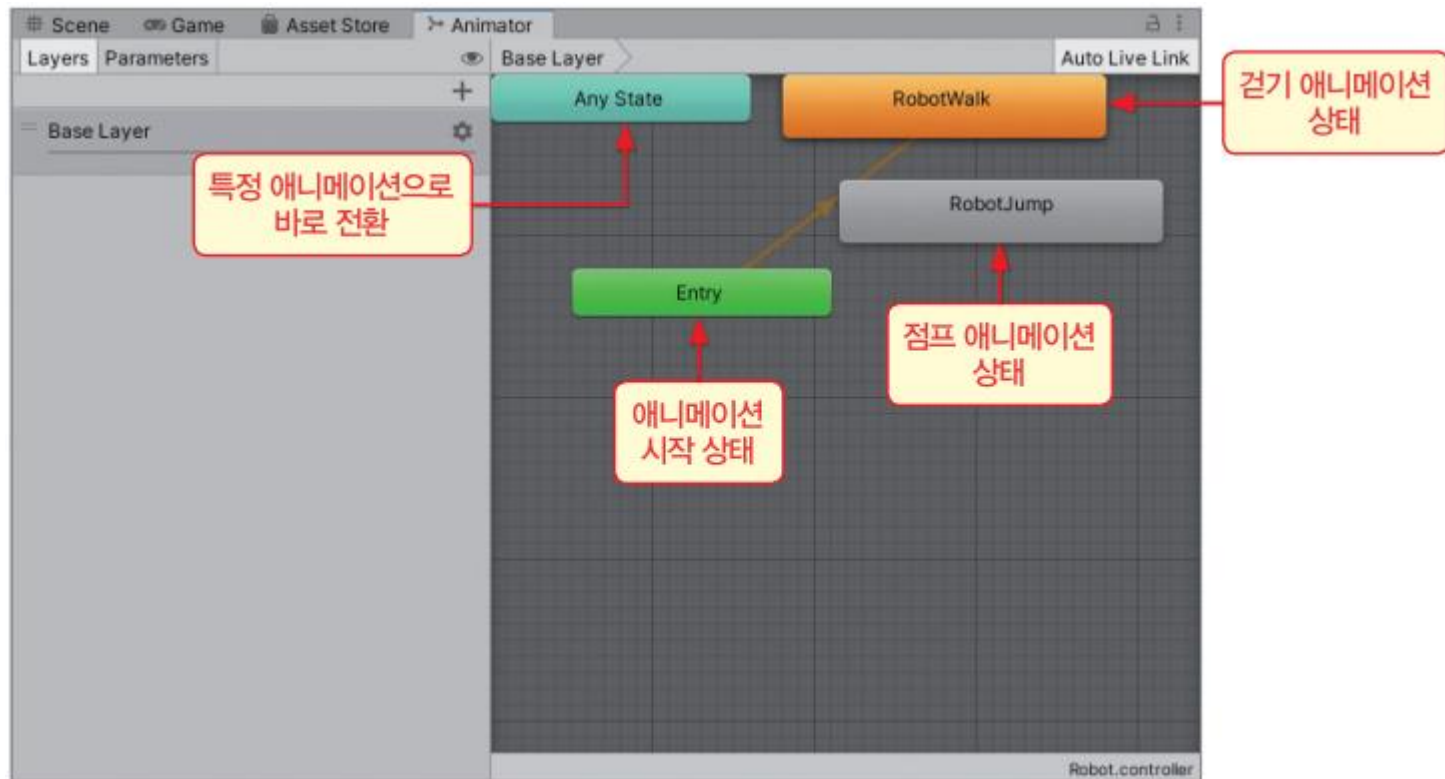
```
01 public class bombController : MonoBehaviour
02 {
03     void OnCollisionEnter2D (Collision2D Col)
04     {
05         GameObject hp = GameObject.Find ("HPControl");
06
07         if (Col.gameObject.tag == "ROBOT")
08         {
09             hp.GetComponent<hpController> ().HpControl ();
10         }
11
12         Destroy (gameObject);
13     }
14 }
```

7. 애니메이션 컨트롤러와 상태 머신

- 점프 애니메이션 만들기
 - [Window]-[Animation]-[Animation]을 선택
 - [RobotWalk]-[Create New Clip]을 선택하여 점프 애니메이션을 생성하고 이름을 RobotJump로 수정
 - 걷기 애니메이션과 같은 방법으로 점프 애니메이션 생성

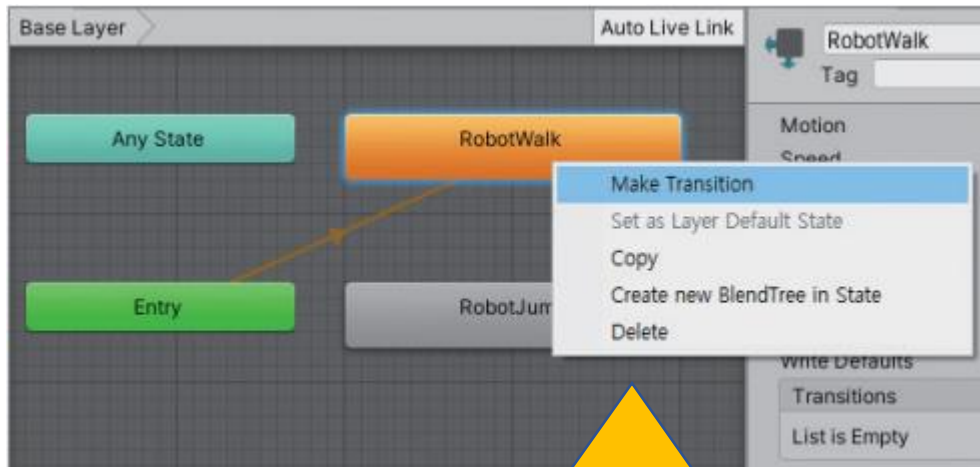
7. 애니메이션 컨트롤러와 상태 머신

- 애니메이터 컨트롤러 설정하기
 - 애니메이션이 여러 개이면 애니메이션 간의 관계를 설정
 - 애니메이터 컨트롤러에서 설정

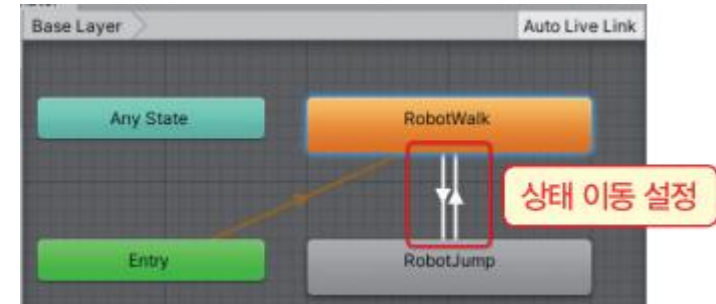


7. 애니메이션 컨트롤러와 상태 머신

- 애니메이터 컨트롤러에서 설정 방법

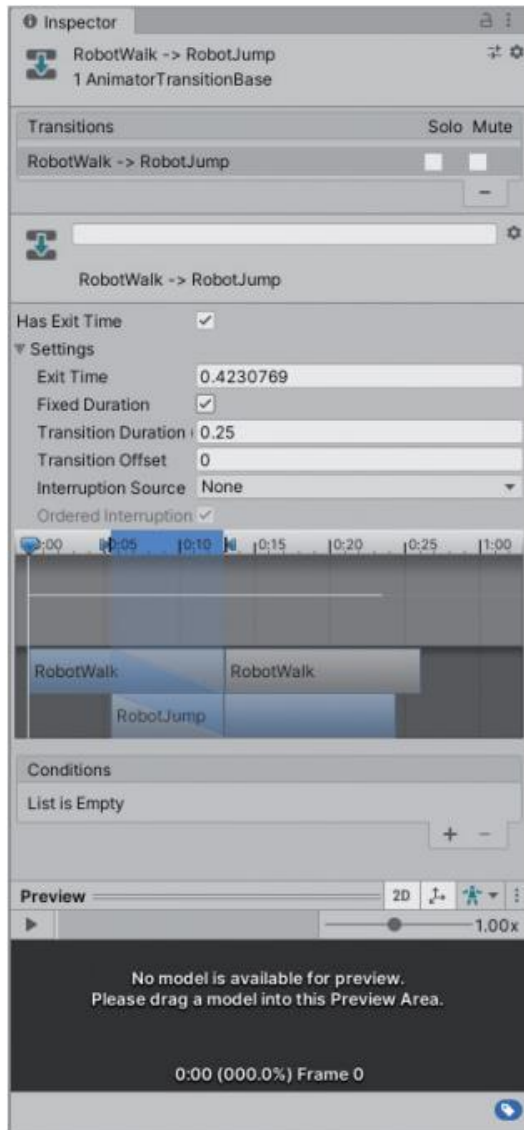


상태에서 마우스 우클릭
팝업창에서 이동 설정가능



상태가 이동 설정

7. 애니메이션 컨트롤러와 상태 머신

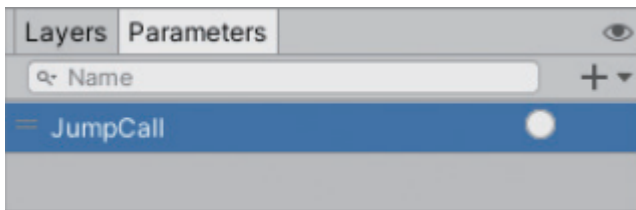
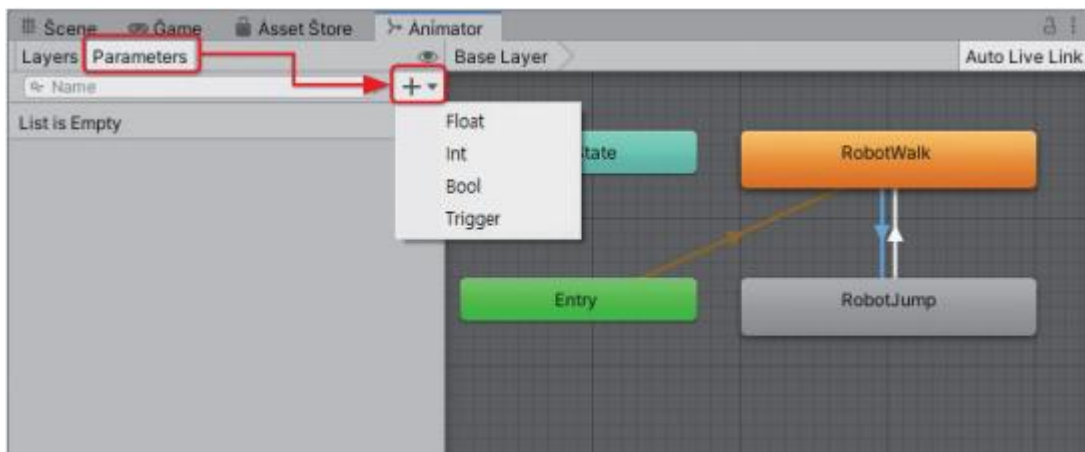


속성	기능
Has Exit Time	이 속성에 체크하면 파라미터에 설정한 값에 따라 즉시 전환하지 않고, Exit Time이 정규화된 시간에 맞추어 전환이 발생하게 한다.
Exit Time	Has Exit Time에 체크되면 Exit Time의 값은 전환을 실행하는 정확한 시간을 나타낸다. 시간을 정규화하여 0.0~1.0으로 설정한다.
Fixed Duration	이 속성에 체크하면 전환 시간이 초로 표시된다.
Transition Duration	현재 상태의 지속 시간을 기준으로 한 상대적인 전환 지속 시간이다. Fixed Duration의 모드에 따라 정규화된 시간 또는 초 단위로 표시된다.
Transition Offset	다음 애니메이션의 플레이를 시작할 시간의 오프셋이다. 예를 들어 이 값이 0.7이면 타임라인의 70% 지점에서 플레이를 시작한다.
Conditions	전환에는 하나 이상의 조건이 있을 수 있다. 조건이 없으면 Exit Time만 고려한다. 전환에 하나 이상의 조건이 있는 경우 전환이 트리거되려면 모든 조건이 충족되어야 한다.

전환 라인 클릭하면
설정가능

7. 애니메이션 컨트롤러와 상태 머신

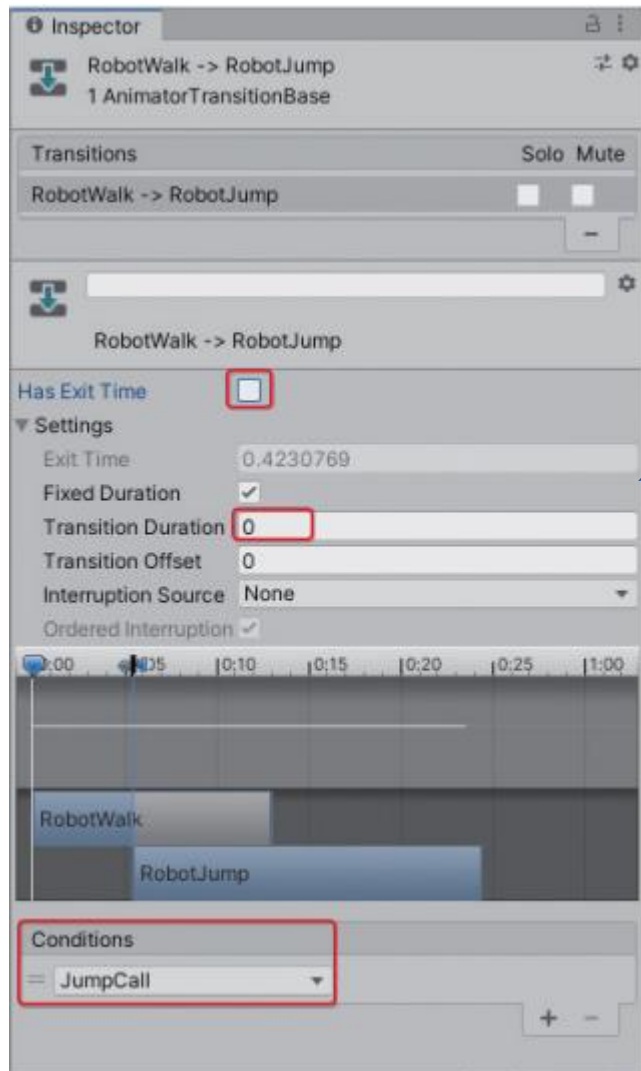
- 동작간 전환하는 조건 설정
 - 전환이 트리거 되려면 특정 조건을 만족해야 하므로 파라미터를 정의하고 Conditions 항목에 설정



Trigger는 초깃값을 true로
설정하면 애니메이션을
플레이한 후 자동으로
파라미터의 값이 false로 설정

7. 애니메이션 컨트롤러와 상태 머신

- 트리거로 인해 이동하도록 걷기→점프로 가는 전환 라인의 속성 설정

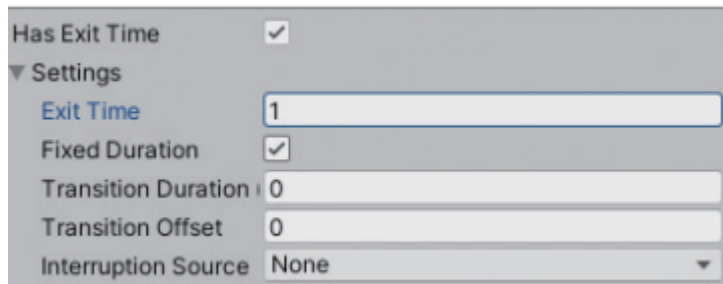


시간적용이 아니므로
-HasExitTime 체크해제
-Transition Duration 0으로

Conditions에
JumpCall 파라미터를 설정

7. 애니메이션 컨트롤러와 상태 머신

- 점프 → 걷기 전환은 점프가 끝나는 즉시 걷기로 전환하도록 설정
 - Has Exit Time 체크, Exit Time은 1, Transition Duration은 0으로 설정

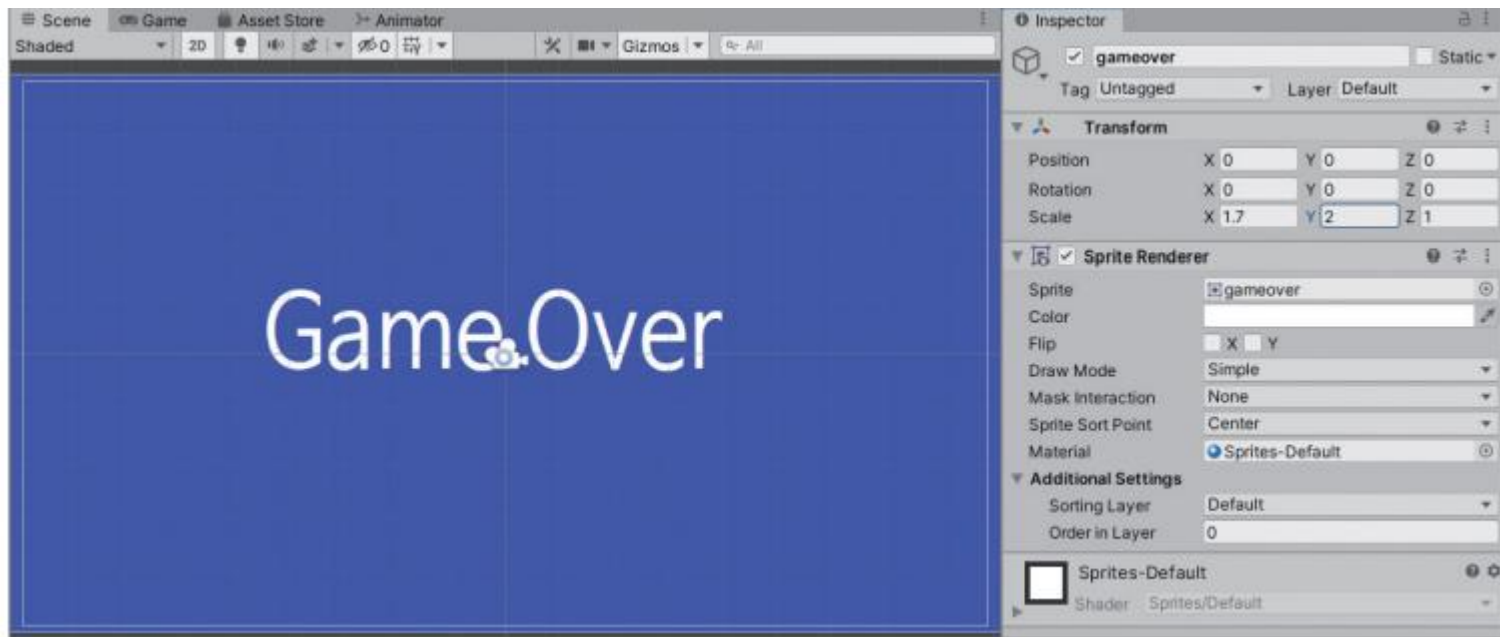
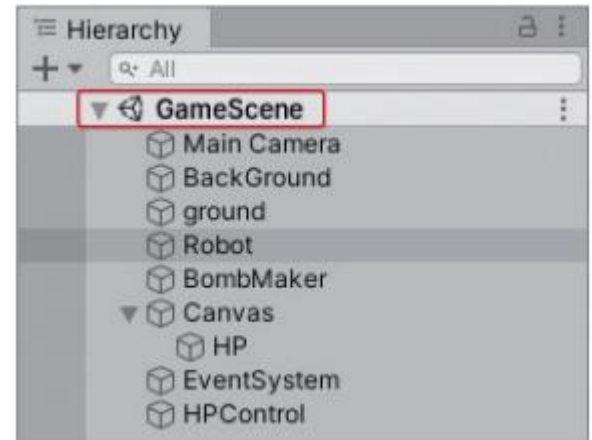


- 스페이스바를 누르면 JumpCall 트리거가 동작하도록 RobotController 스크립트를 수정

```
22 void Update()
23 {
24     float speedx = Mathf.Abs (robotRD.velocity.x);
25
26     if (Input.GetKey (KeyCode.Space) && transform.position.y <= -3.7f ) {
27         animator.SetTrigger ("JumpCall");
28         robotRD.AddForce (transform.up * jumpSpeed);
29     }
30 }
```

8. 씬 전환

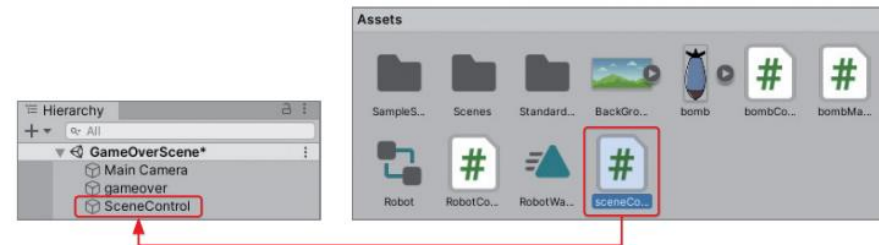
- 게임 씬과 게임 종료 씬 전환 기능
 - 기존 씬은 GameScene으로 저장
 - 파일->New Scene을 선택하여 새로운 씬 생성
 - GameOverScene으로 저장
 - GameOver 이미지를 에셋 폴더로 드래그
 - 씬 창에 GameOver 이미지 배치



8. 씬 전환

- 씬 전환을 위한 스크립트 작성: sceneControl

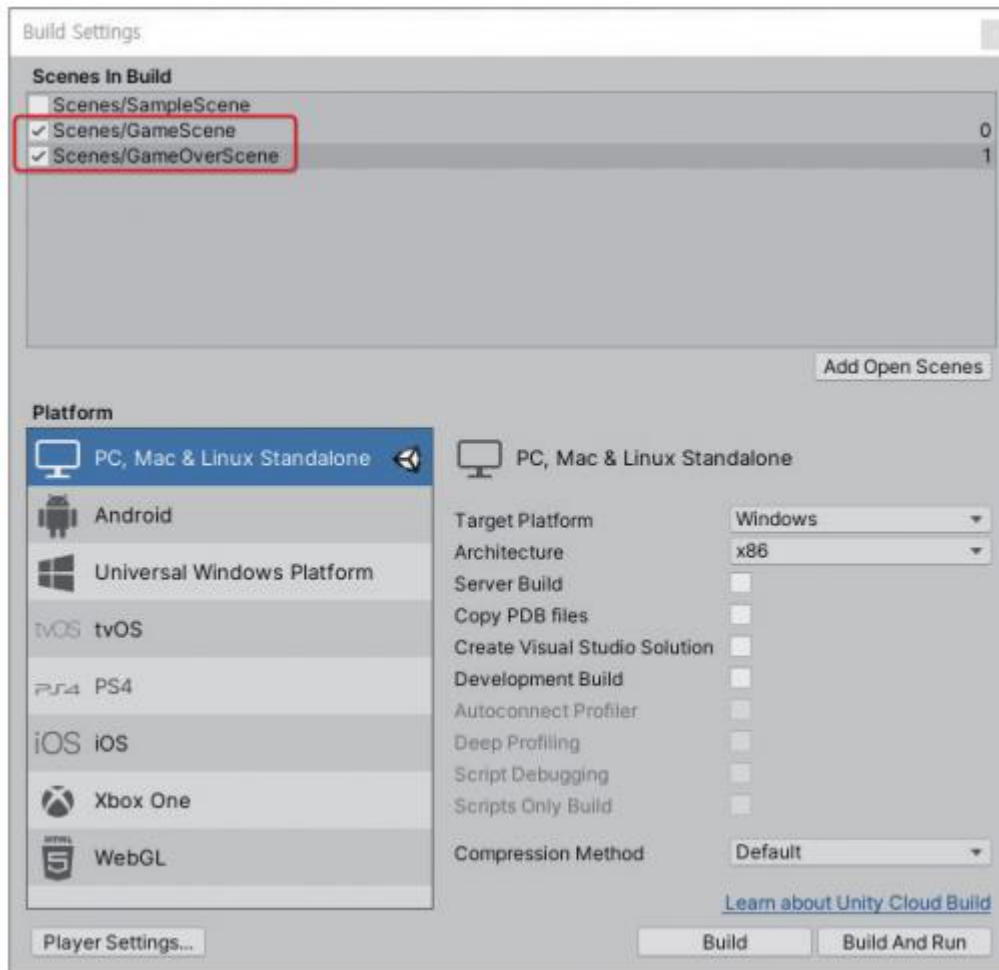
```
01 using System.Collections;
02 using System.Collections.Generic;
03 using UnityEngine;
04 using UnityEngine.SceneManagement;
05
06 public class sceneControl : MonoBehaviour
07 {
08     // Update is called once per frame
09     void Update()
10     {
11         if (Input.GetMouseButtonDown (0))
12         {
13             SceneManager.LoadScene ("GameScene");
14         }
15     }
16 }
```



빈 오브젝트 생성하고
이름 수정 후 스크립트 적용

8. 씬 전환

- 실제 실행에서 씬 전환이 되려면 빌드 설정에 씬을 등록해야함
 - [File]-[BuildSettings]를 선택 -> Build Settings창에서 Scenes in Build에 씬 등록

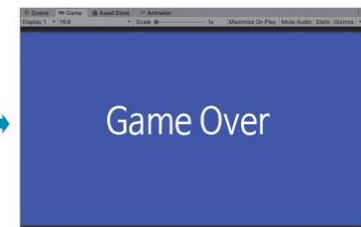


8. 씬 전환

- HP가 0이 되는 순간 GameScene에서 GameOverScene으로 전환하도록 hpController 스크립트 수정

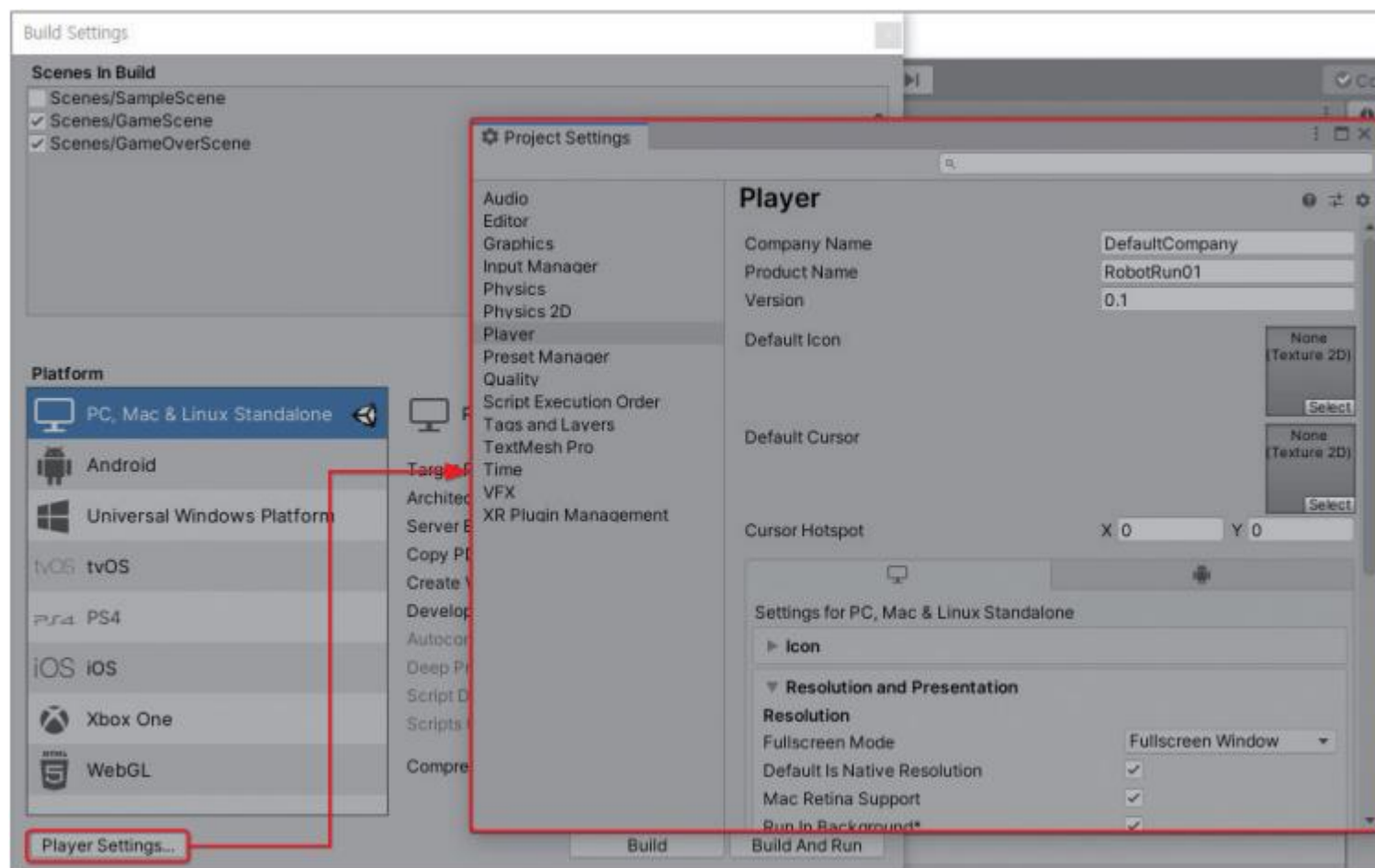
```
01 using System.Collections;
02 using System.Collections.Generic;
03 using UnityEngine;
04 using UnityEngine.UI;
05 using UnityEngine.SceneManagement;
```

```
17 public void HpControl()
18 {
19     hp.GetComponent<Image> ().fillAmount -= 0.05f;
20     if (hp.GetComponent<Image> ().fillAmount <= 0)
21     {
22         SceneManager.LoadScene ("GameOverScene");
23     }
24 }
```



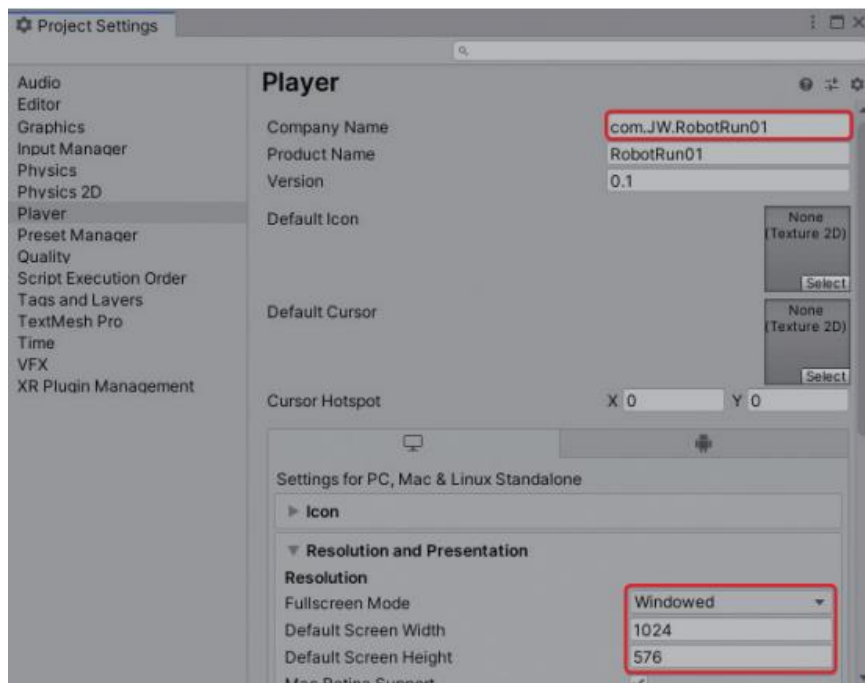
9. 빌드 생성

- 빌드 설정
 - [Build Settings] 창에서 왼쪽 아래의 <Player Settings>클릭



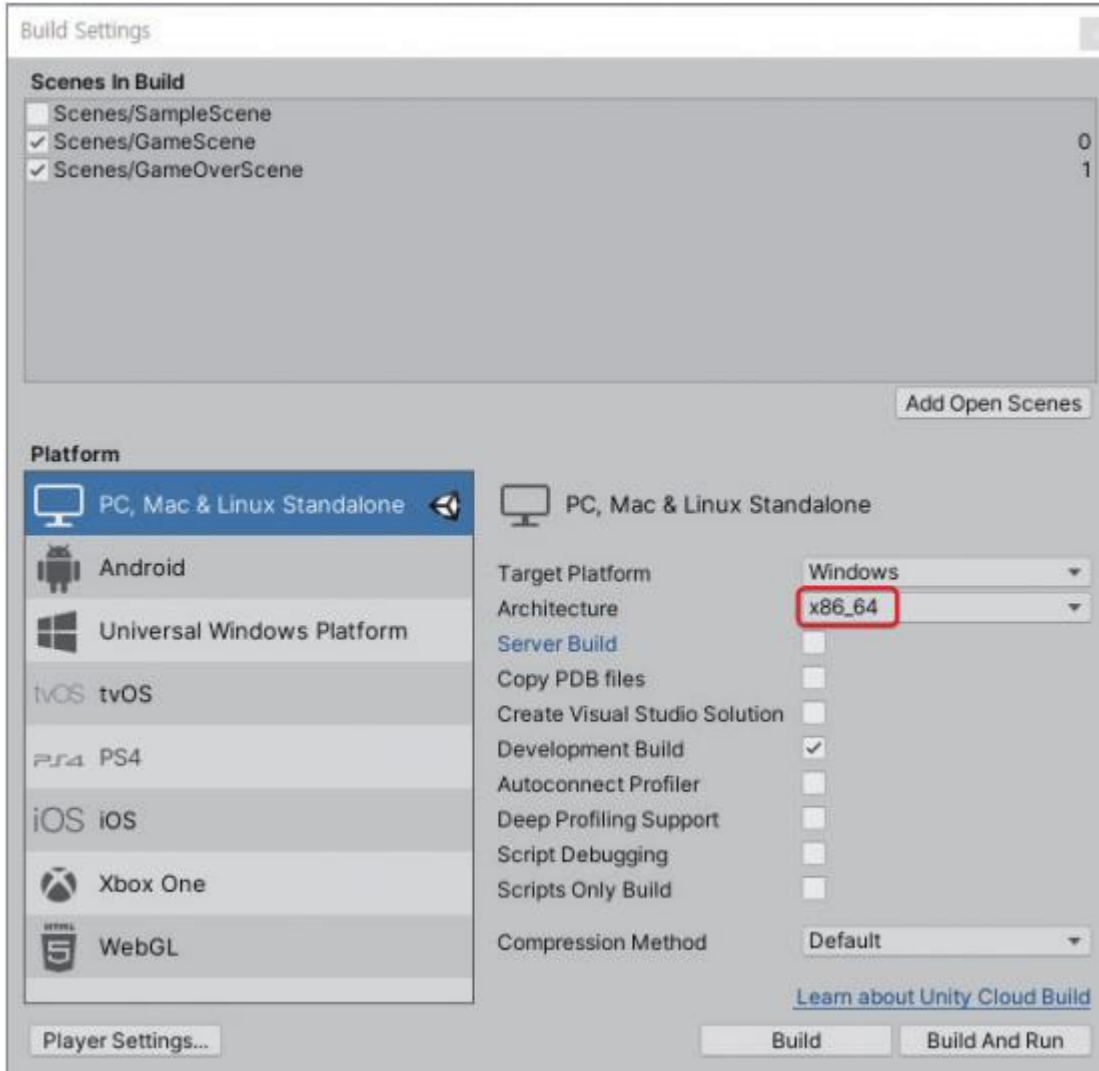
9. 빌드 생성

- [Project Settings] 창의 Player 항목에서 Company Name과 Resolution수정
 - Company Name에 'com.이름.게임명'을 입력
 - Resolution에서는 풀 스크린으로 빌드할 것인지, 윈도 모드로 빌드할 것인지를 선택
 - 윈도 모드를 선택하고 해상도는 16:9에 맞춰서 1024×576으로 설정



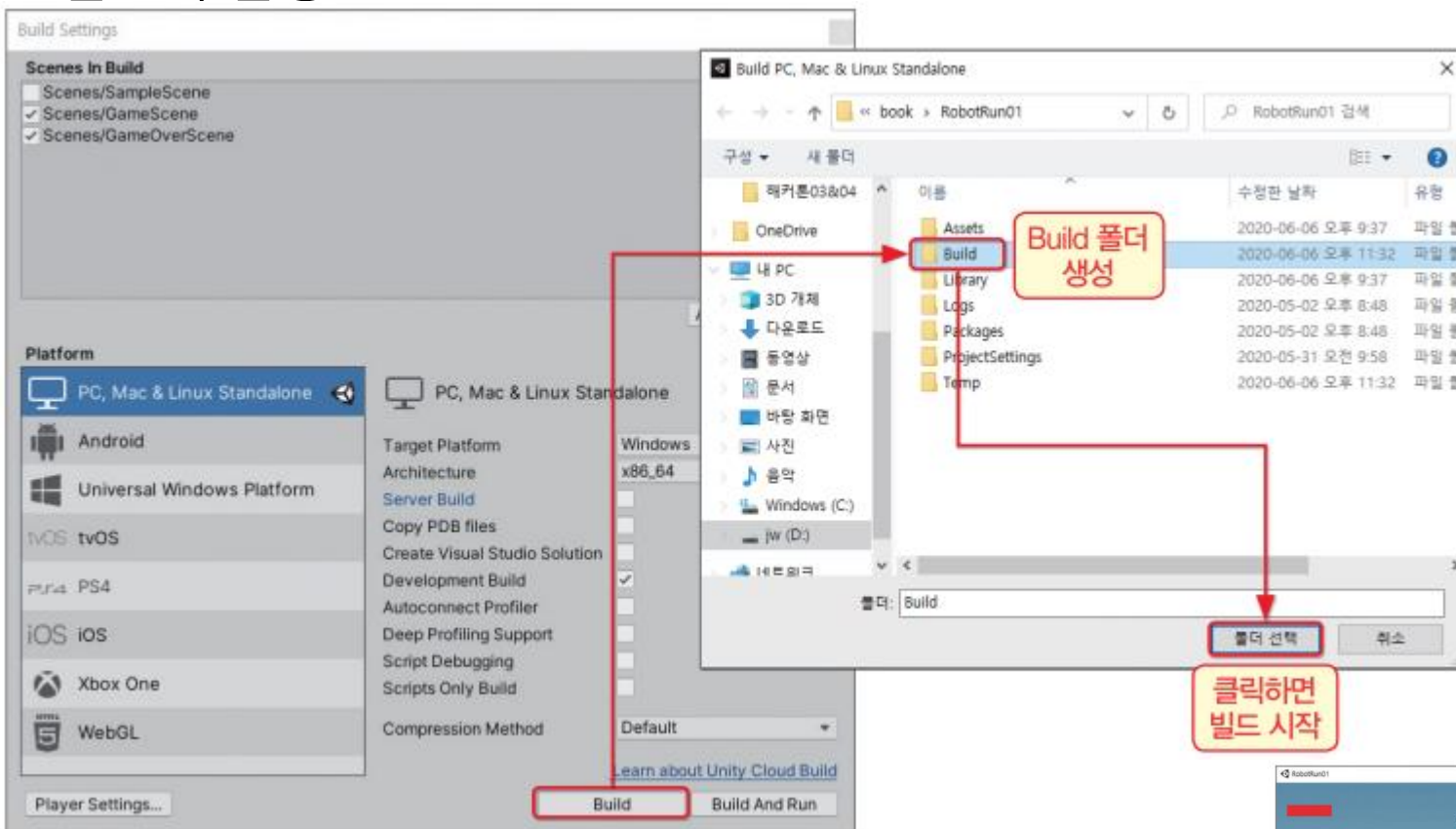
9. 빌드 생성

- Architecture 항목을 x86_64로 설정

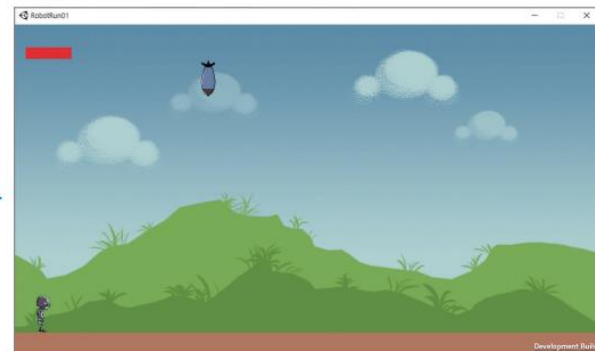
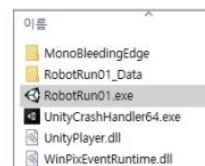


9. 빌드 생성

• 빌드와 실행



exe 파일 실행



- 유니티에서 2D 게임을 만드는 방법을 익힌다.
- 2D 애니메이션 클립을 만드는 방법을 익힌다.
- 애니메이터 컨트롤러를 설정하는 방법을 익힌다.
- 씬을 전환하는 방법을 익힌다.
- 빌드를 만드는 방법을 익힌다.